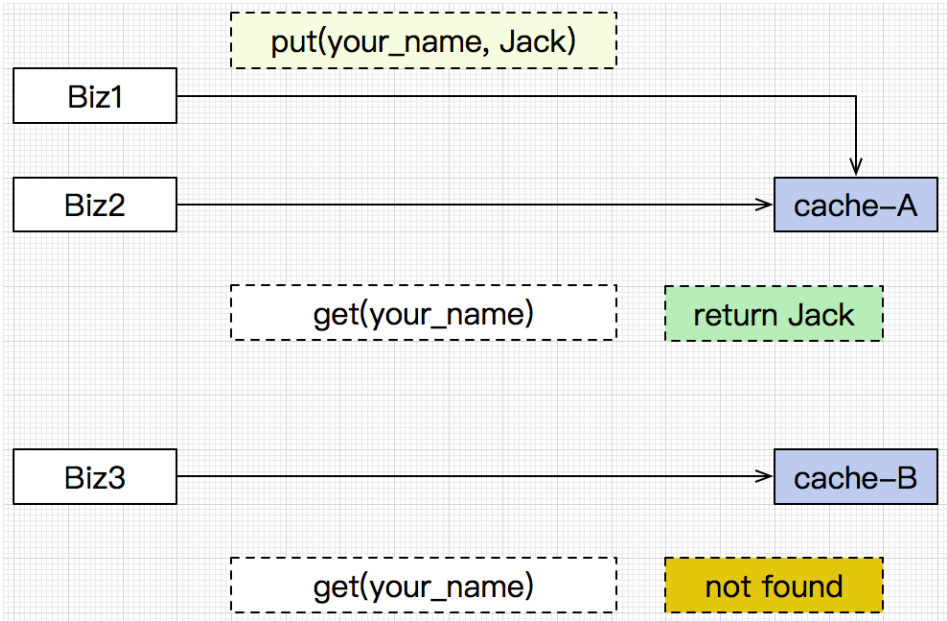


缓存在架构设计中非常重要，对性能优化有明显效果。
缓存默认指单点缓存，即只有 1 个缓存节点。
随着数据量的增加，分布式缓存应运而生，实现大数据量、高负载、高性能、易扩展。

缓存的特点

缓存的基本操作包括读缓存、写缓存。
缓存的数据结构为 Key-Value 格式，Key 可以为字符串、整数等，Value 可以为单值、List、Map、Set 等。
缓存使用 Key 作为索引，为了保证写缓存、读缓存的一致性，需要把 Key 放在固定的位置。



如图：

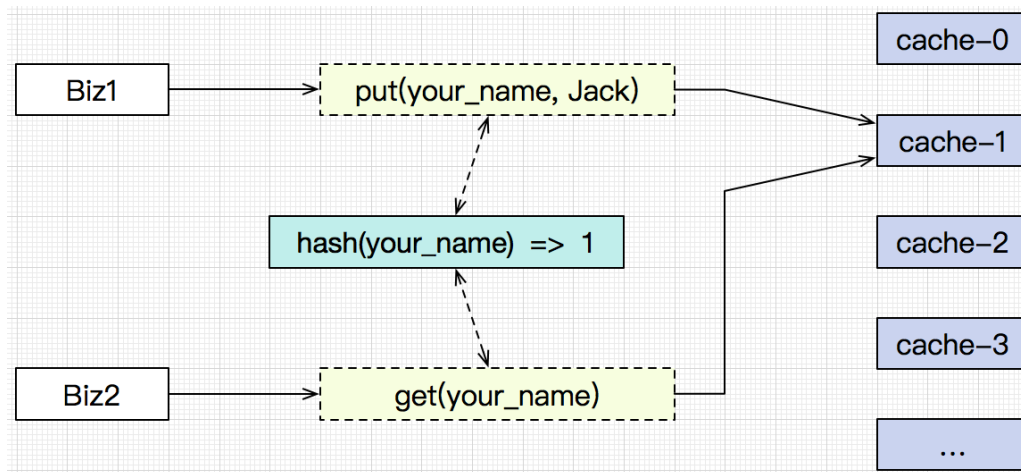
- Biz1 写缓存到 cache-A，Key 为 your_name。
- Biz2 从 cache-A 读缓存，成功获得缓存值。
- Biz3 从 cache-B 读缓存，没有找到缓存值。

缓存实现负载均衡

缓存的读写一致性，要求缓存使用哈希法。

具体规则为：

- 分布式缓存有多个缓存节点，总数为 2 的 N 次方，从 0 开始编号。
- 读写缓存在实际操作前多了一个步骤：找到缓存节点。
- 写缓存，先用 hash 函数计算出缓存节点编号，然后把缓存写到缓存节点。
- 读缓存，先用 hash 函数计算出缓存节点编号，然后从缓存节点查询缓存。



缓存使用负载均衡的示例代码

- 缓存节点的结构。包含缓存机器 IP、编号，存放缓存数据的集合。

```

@Getter
@Setter
@Accessors(chain = true)
public class CacheNode {
    public String serverIP;
    public int index;// 编号
    public TreeMap<String, String> cacheMap = new TreeMap<>();// 缓存数据
}

```

- 初始化缓存节点。为缓存节点生成编号，从 0 开始，之后不会变化。

```

// 构建缓存节点
public static List<CacheNode> buildNodeList() {
    List<CacheNode> list = new ArrayList<>();
    list.add(new CacheNode().setIndex(0).setServerIP("192.168.1.100"));
    list.add(new CacheNode().setIndex(1).setServerIP("192.168.1.101"));
    list.add(new CacheNode().setIndex(2).setServerIP("192.168.1.102"));
    list.add(new CacheNode().setIndex(3).setServerIP("192.168.1.103"));
    return list;
}

```

- 负载均衡使用哈希法。哈希值需要尽量均匀。

```

// 哈希法。
public static CacheNode balanceByHash(List<CacheNode> nodeList, String cacheKey) {
    String salt = "today";// 加盐
    String input = cacheKey + salt;
    // 计算 hash。
    int hash = 17;
    int mul = 15;
    for (int m = 0; m < input.length(); m++) {
        hash = hash * mul + input.charAt(m);
    }
}

```

```

    }
    hash = Math.abs(hash);
    // 取节点
    int index = hash % nodeList.size();
    CacheNode node = nodeList.get(index);
    return node;
}

```

- 批量写缓存。先使用负载均衡函数找到缓存节点，然后写缓存。

```

// 写缓存
public static void batchPutCache(List<CacheNode> nodeList) {
    List<String> foodList = Arrays.asList("apple", "orange", "banana");
    for (String food : foodList) {
        for (int m = 0; m < 9000; m++) {
            String cacheKey = food + "_" + m;
            String cacheValue = cacheKey + " is good";
            // 找到缓存节点
            CacheNode node = balanceByHash(nodeList, cacheKey);
            // 写缓存
            node.getCacheMap().put(cacheKey, cacheValue);
        }
    }
    // 统计
    String format = "%-15s  %-6s  %-10s  \n";
    System.out.printf(format, "serverIP", "index", "cacheCount");
    for (CacheNode node : nodeList) {
        System.out.printf(format, node.serverIP, node.index, node.cacheMap.size());
    }
    System.out.println();
}

```

- 读缓存。先使用负载均衡函数找到缓存节点，然后读缓存。

```

// 读缓存
public static void getCache(List<CacheNode> nodeList, String cacheKey) {
    // 找到缓存节点
    CacheNode node = balanceByHash(nodeList, cacheKey);
    // 读缓存
    String cacheValue = node.getCacheMap().get(cacheKey);
    System.out.println("getCache key = " + cacheKey + " value = " + cacheValue);
}

```

- 综合测试。

```

public static void main(String args[]) {
    List<CacheNode> nodeList = buildNodeList();
    batchPutCache(nodeList);
    getCache(nodeList, "apple_33");
}

```

```
getCache(nodeList, "orange_156");
getCache(nodeList, "banana_789");
}
```

● 测试结果。

查看每个缓存节点的缓存数量，缓存分布比较均匀。
读缓存 3 次，都读到缓存值，说明缓存读写功能正常。

serverIP	index	cacheCount
192.168.1.100	0	6750
192.168.1.101	1	6751
192.168.1.102	2	6750
192.168.1.103	3	6749

```
getCache key = apple_33 value = apple_33 is good
getCache key = orange_156 value = orange_156 is good
getCache key = banana_789 value = banana_789 is good
```

引申思考

如果缓存的负载均衡使用随机法，会产生什么结果？

缓存节点的数量为什么是 2 的 N 次方？如果是 99 个节点，有什么结果？

这里的例子使用多个机器作为缓存节点，某些单机缓存也需要负载均衡。你能找出一些例子吗？